

Cours magistral de securite embarquee et materielle

Un support structure comme un vrai cours de master : definitions, modeles de menace, formules, methodes d'attaque, schemas d'analyse, exemples concrets et contre-mesures pour AES, DES, RSA, ECC et les systemes embarques modernes.

Master Securite des
Systemes Embarques

Universite Cheikh Anta Diop

PDF du dossier
doc_linuc_embarquee +
traces du workspace

Plan du cours

1. Vision generale
2. Cadre formel et vocabulaire
3. Modele de menace materiel
4. Bases crypto pour embarque
5. DES, AES et modes
6. RSA, DH, ECC et signatures
7. Side-channel attacks
8. Methodologie complete d'une attaque
9. SPA, DPA, CPA et templates
10. Fault injection
11. Debug, boot et secrets
12. Secure boot et mise a jour
13. PUF, TRNG et stockage de cles
14. FPGA, ASIC, DFT et scan chains
15. Hardware trojans
16. Linux embarque et supply chain
17. Contre-mesures globales
18. Projets sans materiel
19. Checklist d'audit
20. Notes de lecture PDF

1. Vision generale de la securite materielle

La securite materielle etudie ce qui se passe quand la cryptographie, le logiciel et les secrets vivent dans un objet physique : microcontroleur, SoC, FPGA, carte bancaire, objet IoT, routeur, telephone, module medical, calculateur automobile ou carte de developpement.

Un algorithme peut etre mathematiquement robuste et son systeme quand meme vulnerable. La faille peut venir d'un temps d'execution variable, d'une consommation electrique dependante des bits secrets, d'un port JTAG mal verrouille, d'une cle stockee en clair dans la flash, d'un bootloader qui accepte un firmware non signe, d'un FPGA dont le bitstream est copiable, ou d'une dependance logicielle compromise dans la chaine de build.

Objectif defensif

Concevoir un systeme qui protege ses secrets, son code, son execution et son identite meme face a un attaquant local ou physique.

Objectif offensif ethique

Mesurer les fuites, demontrer les risques sur banc autorise, puis proposer des corrections testables.

Frontiere importante

La securite ne s'arrete pas a l'algorithme. Elle inclut implementation, packaging, fabrication, debug, boot, mise a jour, exploitation et maintenance.

Les grandes surfaces d'attaque

Surface	Question de securite	Exemple d'attaque
Physique	Que peut observer ou modifier un attaquant avec acces a la carte ?	Mesure de puissance, sonde EM, glitch tension/horloge, decapsulation.
Microarchitecture	Le CPU fuit-il via cache, branchements, speculation, TLB ?	Flush+Reload, Prime+Probe, timing sur tables AES.

Surface	Question de securite	Exemple d'attaque
Firmware	Le code est-il authentifie, chiffre, anti-rollback ?	Injection d'un firmware ancien ou modifie.
Debug	JTAG/SWD/UART restent-ils accessibles ?	Dump flash, halt CPU, lecture RAM, modification de boot.
Fabrication	Peut-on inserer, remplacer ou cloner un composant ?	Hardware trojan, counterfeit IC, IP piracy.
Logiciel embarque	Les dependances et outils de build sont-ils fiables ?	Compromission d'une bibliotheque transitive ou d'un paquet.

2. Cadre formel et vocabulaire de la securite materielle

Pour travailler proprement, on distingue trois niveaux. Le premier est l'algo mathematique, par exemple AES ou RSA. Le deuxieme est l'implementation, par exemple un code C, une RTL ou un microcode. Le troisieme est le systeme physique, c'est-a-dire la puce, le PCB, l'alimentation, les capteurs, les bus et les interfaces de debug.

Algorithme

Implementation

Plateforme physique

Fuite observable

Modele de fuite

On note souvent la fuite mesuree sur une trace au temps t par une fonction du type :

$$L(t) = a \cdot f(x, k, t) + b + n(t)$$

ou x est le texte clair ou le challenge, k la cle, f la quantite intermediaire sensible, a un facteur d'echelle, b un offset et $n(t)$ le bruit de mesure.

Trois grandeurs utiles

Grandeur	Definition	Utilite
SNR	Rapport signal sur bruit entre la partie utile et la variance parasite.	Comparer deux configurations ou deux contre-mesures.

Grandeur	Definition	Utilite
Correlation	Mesure du lien entre la prediction de fuite et les traces observees.	Base de la CPA et du classement des hypotheses.
t-test TVLA	Test statistique pour verifier si deux ensembles de traces sont distinguables.	Evaluer si une implementation fuit potentiellement.

$$r = \text{cov}(X, Y) / (\sigma_X \cdot \sigma_Y)$$

$$t = (\mu_1 - \mu_2) / \text{sqrt}(s_1^2/n_1 + s_2^2/n_2)$$

Lecture d'une attaque

1. On choisit un point sensible : par exemple la sortie de la S-box AES ou une multiplication scalaire ECC.
2. On calcule une variable intermediaire candidate pour chaque hypothese de cle.
3. On compare la prediction au signal mesure avec correlation, difference de moyennes ou classification.
4. On retient les hypotheses dominantes puis on valide sur un second lot de traces.

Exemple simple : pour AES, une hypothese sur un octet de cle produit une valeur intermediaire $v = \text{Sbox}[p \text{ XOR } k]$. Si la consommation suit une distance ou une quantite proportionnelle aux bits a 1 de v , la bonne hypothese remonte statistiquement plus haut que les autres.

2. Modele de menace materiel

Un modele de menace de securite materielle doit dire qui attaque, ce qu'il possede, ce qu'il sait, combien de temps il a, et ce qu'il peut detruire. Sans cela, le mot "securise" ne veut rien dire.

Niveaux d'attaquant

Niveau	Capacites	Exemples	Defense attendue
Distant	Paquets reseau, API, mises a jour	Firmware malveillant, exploitation service	Authentification, chiffrement, secure update
Local logiciel	Processus sur meme machine	Cache attacks, escalation de privileges	Isolation, constant-time, durcissement OS
Local physique simple	Acces aux connecteurs, flash externe, UART	Dump SPI flash, console root	Secure boot, chiffrement flash, verrouillage debug
Laboratoire	Oscilloscope, sonde EM, glitcher, microscope	DPA, EMFI, voltage glitch	Masking, detection de fautes, blindage, tests TVLA
Invasif	Decapsulation, microprobing, FIB	Lecture bus interne, modification metal	Tamper resistance, zeroization, packaging securise
Supply chain	Fournisseur, fab, dependance, build system	Hardware trojan, paquet compromis	SBOM/HBOM, provenance, audits, tests, diversification

Principe utile : plus l'attaquant est proche physiquement, plus il faut proteger les secrets contre l'observation et la perturbation, pas seulement contre la lecture logique.

Actifs a proteger

- Clefs cryptographiques : cle de boot, cle de chiffrement flash, cle TLS, cle ECDSA de signature.
- Firmware et configuration : bootloader, kernel, device tree, fichiers de calibration, politiques de securite.
- Identite de l'equipement : certificats, serial, identifiants de production.
- Propriete intellectuelle : bitstream FPGA, RTL, microcode, algorithmes proprietaires.
- Disponibilite et surete : le systeme ne doit pas seulement cacher un secret, il doit rester dans un etat sur.

3. Bases cryptographiques utiles en embarque

Le livre *Understanding Cryptography* de Christof Paar et Jan Pelzl donne la base mathématique et pratique : chiffrement symétrique, chiffrement asymétrique, signatures, fonctions de hachage et établissement de clés. En sécurité matérielle, on ajoute une question : comment ces primitives sont-elles implémentées dans une puce réelle ?

Primitives essentielles

Primitive	Role	Usage embarque	Risque d'implémentation
AES	Chiffrement symétrique bloc	Flash encryption, TLS, stockage, bus sécurisé	CPA/DPA sur S-box, cache attacks sur tables
Hash SHA-2/SHA-3	Empreinte, intégrité	Mesure de firmware, signatures, KDF	Erreur de mode, collision si algo obsolète
HMAC/CMAC/GMAC	Authentification de message	Secure update, messages CAN/IoT	Comparaison non constante, clé mal protégée
RSA/ECC	Signature, échange de clés	Secure boot, TLS, attestation	Timing, fautes RSA-CRT, nonce ECDSA fuyant
TRNG/DRBG	Alea cryptographique	Nonce, clé, masques side-channel	Alea faible, health tests absents

Règles de conception

- Ne jamais inventer son propre algorithme cryptographique pour un produit réel.
- Séparer les usages de clés : une clé de boot ne doit pas servir au chiffrement données.
- Authentifier avant d'exécuter : un firmware chiffre mais non signé peut rester dangereux.
- Protéger les clés au repos, en transit et pendant le calcul.
- Choisir les modes modernes : GCM, CCM, SIV ou encrypt-then-MAC selon le contexte.

4. DES, AES et modes d'operation

DES : utile pour apprendre, obsolete pour proteger

DES est un chiffrement par bloc de 64 bits construit comme un reseau de Feistel a 16 tours. Il illustre tres bien les notions de confusion et diffusion : les S-boxes creent la non-linearite, les permutations diffusent les bits. Sa cle effective de 56 bits est aujourd'hui beaucoup trop courte, ce qui explique son abandon pour les usages modernes.

Apport du livre de Paar/Pelzl : DES sert de modele pedagogique pour comprendre Feistel, S-boxes, key schedule, recherche exhaustive, cryptanalyse lineaire/differentielle et remplacement progressif par AES/3DES.

AES : standard moderne, mais implementation sensible

AES chiffre des blocs de 128 bits avec des cles de 128, 192 ou 256 bits. Sa structure est un reseau substitution-permutation : `SubBytes` , `ShiftRows` , `MixColumns` , `AddRoundKey` . Pour les attaques side-channel, le point le plus classique est la sortie de la S-box du premier tour :

Valeur cible CPA AES : $V_i(k) = \text{Sbox}[P_i \text{ XOR } k]$. Pour chaque octet de cle, on teste 256 hypotheses et on cherche celle dont le modele de fuite correspond le mieux aux traces mesurees.

Modes d'operation

Mode	Propriete	Usage	Attention
ECB	Chiffre chaque bloc independamment	Presque jamais recommande	Revele les motifs, substitution de blocs
CBC	Chaine les blocs avec IV	Legacy, stockage ancien	Besoin de padding sur, MAC separe

Mode	Propriete	Usage	Attention
CTR	Transforme AES en flux	Embarque rapide, parallelisable	Nonce/compteur jamais reutilise
GCM	Confidentialite + integrite	TLS, stockage, messages	Nonce unique obligatoire
CCM	CTR + CBC-MAC	IoT, BLE, 802.15.4	Parametrage plus contraignant

Erreur classique : chiffrer un firmware sans signature ne suffit pas. Le chiffrement cache le contenu, mais seule l'authentification dit que le code vient d'une autorite legitime et n'a pas ete modifie.

5. RSA, Diffie-Hellman, ECC et signatures

RSA

RSA repose sur la difficulté de factoriser un module $N = p \cdot q$. En embarqué, RSA est surtout utilisé pour les signatures de firmware ou TLS legacy. Le calcul coûte cher : exponentiation modulaire sur grands entiers, optimisations CRT, Montgomery, fenêtrage. Ces optimisations sont utiles, mais elles ouvrent aussi des surfaces timing et fault injection.

Diffie-Hellman et ECDH

Diffie-Hellman permet d'établir une clé de session sur un canal public. Sa version elliptique, ECDH, est très populaire en embarqué car elle offre une sécurité forte avec des clés plus courtes que RSA classique. Mais l'authenticité reste nécessaire : sans certificats, clé publique préinstallée ou attestation, l'échange est vulnérable à l'homme-du-milieu.

ECC et ECDSA

ECC manipule des points sur courbe elliptique. L'opération centrale est la multiplication scalaire $Q = kP$. Si l'implémentation double-and-add fuit la différence entre doublement et addition, le scalaire secret peut se révéler. Pour ECDSA, le nonce de signature est critique : s'il est réutilisé, biaisé ou partiellement fuyant, la clé privée peut être retrouvée.

Primitive	Operation sensible	Attaques	Contre-mesures
RSA	Exponentiation, CRT	Timing, SPA, fault RSA-CRT	Blinding, constant-time, vérification signature avant sortie
DH	Exponentiation modulaire	Timing, petits sous-groupes, MITM	Vérification paramètres, blinding, certificats
ECC/ECDH	Multiplication scalaire	SPA, timing, cache, fautes	Montgomery ladder, formules unifiées, validation de points
ECDSA	Nonce éphémère	Nonce reuse, nonce leakage, fault	RFC 6979 ou TRNG solide, protection du calcul, vérification

6. Side-channel attacks

Une attaque side-channel n'attaque pas l'algorithme abstrait. Elle observe une fuite produite par son implementation : temps, puissance, rayonnement electromagnetique, cache, bruit, temperature, erreurs ou comportement reseau.

Secret

Calcul physique

Fuite mesurable

Inference statistique

Familles

Famille	Signal observe	Exemple	Cible typique
Timing	Duree d'une operation	RSA non constant-time	Exposant, cle, branche secret
Power	Courant/alimentation	CPA AES	Octets de cle
EM	Rayonnement local	Sonde proche de la puce	Operation localisee
Cache	Temps d'accès memoire	Flush+Reload sur tables	Index secret, S-box
Acoustique/thermique/optique	Bruit, chaleur, LED	Fuite visuelle ou capteur	Activite, etat interne

Timing attacks selon Kocher

Le papier de Kocher sur les timing attacks montre qu'un attaquant peut retrouver des informations sur les exposants secrets de RSA, Diffie-Hellman ou DSS si le temps d'exécution varie selon les bits du secret. L'attaque se formalise comme un problème de détection de signal : le temps contient une petite composante liée au secret et beaucoup de bruit. Avec assez de mesures, la composante utile devient visible.

Leçon importante : ajouter des délais aléatoires augmente le nombre de mesures nécessaires, mais ne supprime pas la fuite. Une vraie défense timing cherche l'exécution constante et l'absence d'accès mémoire dépendant du secret.

6bis. Methodologie complete d'une attaque side-channel

Une attaque side-channel serieuse suit presque toujours la meme logique. On ne commence pas par "casser la cle", on commence par comprendre ou la puce fuit, combien elle fuit, et sous quelle forme statistique cette fuite devient exploitable.

Cycle d'attaque

1. Cibler

Choisir l'algorithme, l'etape interne et la cle visee : AES premier tour, RSA-CRT, multiplication scalaire ECC, verification de signature, etc.

2. Observer

Capturer des traces ou des timings en gardant les parametres stables : textes clairs, frequences, synchronisation, temperature et alimentation.

3. Nettoyer

Aligner les traces, retirer les derives, filtrer le bruit, normaliser et verifier que le point d'interet est visible.

4. Modeler

Choisir le modele de fuite : Hamming Weight, Hamming Distance, timing variable, cache access, classe statistique, template, ou modele appris.

5. Tester

Calculer la statistique d'attaque et classer les hypotheses de cles jusqu'a obtenir un rang 1 stable et reproductible.

6. Valider

Verifier sur un autre lot de traces, mesurer la robustesse, et documenter les limites, le taux de succes et le nombre de traces necessaires.

Questions a poser avant l'attaque

Question	Pourquoi elle compte	Impact sur la methode
Qu'est-ce qui fuit exactement ?	Le bon point de fuite determine toute l'efficacite.	On choisit la bonne fenetre temporelle, le bon modele et le bon filtrage.
La fuite est-elle sur la valeur ou sur les transitions ?	Le modele HW ou HD n'est pas toujours le bon.	On adapte le calcul de prediction et la statistique associee.
Le systeme est-il synchronise ?	Un mauvais alignement peut masquer une fuite forte.	On ajoute des techniques d'alignement et de resynchronisation.
Le protocole est-il actif ou passif ?	Une attaque par faute ne se traite pas comme une attaque passive.	On choisit entre observation, perturbation ou combinaison des deux.

Modeles de fuite les plus utiles

Modele	Idee	Quand il est pertinent
Hamming Weight	Le cout depend du nombre de bits a 1.	Registres, bus, sorties de S-box, operations arithmetiques.
Hamming Distance	Le cout depend du nombre de bits modifies entre deux etats.	Transitions de registre, caches, bus et memoires internes.
Timing	La duree varie selon les donnees ou le secret.	Branchements secrets, divisions, mod operations, lookup non constant-time.
Template / profilage	On apprend des classes de traces avant d'attaquer la cible.	Quand on peut profiler un appareil similaire ou identique.
Higher-order	On combine plusieurs points de fuite masques.	Implementations masquees ou partagees en plusieurs composants.

Attaques possibles selon le type de fuite

Type de fuite	Attaques typiques	Exemple concret
Temps	Timing attacks, cache timing, branch prediction leakage	RSA non constant-time, comparaison de tags, lookup table AES.
Puissance	SPA, DPA, CPA, template, higher-order DPA	AES sur microcontrôleur, DES historique, ECC scalaire.
EM	EM analysis, EM-DPA, EM profiling	Sonde de proximité sur une puce ou un module sécurisé.
Cache	Prime+Probe, Flush+Reload, Evict+Time	Fonctions cryptographiques table-based sur CPU partagé.
Fautes	Glitch, DFA, instruction skip, fault-assisted key recovery	RSA-CRT, AES derniers tours, boot verification bypass.

Lecture algorithmique : ou attaquer AES, DES, RSA et ECC

Algorithme	Point sensible	Attaques fréquentes	Message de défense
AES	Premier tour, S-box, MixColumns, key schedule	CPA, DPA, template, EM, cache timing, DFA fin de tours	Constant-time, masking, shuffling, AES-NI, verification TVLA
DES	S-box, Feistel rounds, key schedule	DPA historique, timing sur implementations naïves, attaque par tables	Ne plus l'utiliser pour la sécurité, migrer vers AES ou autre standard moderne
RSA	Exponentiation modulaire, CRT, comparaison finale	Timing, SPA, blinding failure, fault CRT, cache sur Montgomery	Blinding, constant-time, verification du résultat, protections contre fautes
ECC	Multiplication scalaire, addition de points, nonce ECDSA	SPA, timing, template, lattice attacks si nonce fuit, fault sur signature	Ladder, formules unifiées, point validation, nonce deterministic et protégée

Mesures utiles en evaluation : rank de cle, success rate, guessing entropy, SNR, TVLA t-test, trace-to-success ratio. Une defense n'est credible que si elle ameliore ces indicateurs face a l'attaque visee.

Projet sans materiel et sans zone grise

Si tu n'as pas de banc de mesure, le plus propre est de travailler avec des jeux de traces publics, des exemples pedagogiques et les traces deja presentes dans ton workspace. Cela permet de faire un vrai rapport de securite materielle sans te bloquer sur l'acquisition.

1. Utiliser un dataset public type ASCAD, DPA Contest, ChipWhisperer ou des jeux de traces pedagogiques fournis par les enseignants.
2. Faire une attaque CPA ou template sur AES pour montrer la mecanique complete de la fuite jusqu'a la cle.
3. Faire une experience de detection hardware trojan avec les traces IEEE dataport deja disponibles dans le workspace, pour relier side-channel et surveillance d'integrite.
4. Documenter l'environnement, les hypotheses, la taille d'echantillon, les limites et les resultats reproductibles.

Important : je n'ai pas integre de procedure illicite ou de source douteuse. Pour un memoire UCAD, les traces publiques et les donnees de laboratoire sont la bonne voie : elles sont defendables, citables et exploitables dans un cadre academique.

7. SPA, DPA, CPA et template attacks

SPA : Simple Power Analysis

La SPA consiste à interpréter directement une ou quelques traces. Dans une exponentiation square-and-multiply naïve, un bit 1 peut produire une séquence "square + multiply", tandis qu'un bit 0 produit seulement "square". Si les formes sont visibles, l'exposant se lit presque comme un code barre.

DPA : Differential Power Analysis

Le papier de Kocher, Jaffe et Jun montre que l'on peut attaquer DES et d'autres algorithmes en séparant les traces selon une prédiction de bit intermédiaire. Le bon sous-cle crée une différence moyenne au moment où la puce manipule la valeur cible.

DPA : choisir une fonction de sélection $D(C, b, K_s)$. Pour chaque hypothèse de sous-cle, séparer les traces en deux groupes selon $D = 0$ ou $D = 1$, puis calculer la différence de moyennes. Le bon K_s produit un pic exploitable.

CPA : Correlation Power Analysis

Le papier de Brier, Clavier et Olivier formalise le modèle de fuite par distance de Hamming. Une puce consomme une partie dépendante des transitions de données, souvent modélisée par $H(D \text{ XOR } R)$, où R est un état de référence inconnu. La CPA calcule la corrélation entre fuite prédite et trace observée.

Modèle CPA : $W = a * H(D \text{ XOR } R) + b$. W est la consommation, a un gain, b le bruit et les composantes indépendantes, D la donnée manipulée, R l'état de référence.

```
# CPA AES simplifiée : un octet de cle
for k in range(256):
    prediction = HW(SBOX[plaintext[:, byte] ^ k])
    for t in range(nb_samples):
        score[k, t] = pearson(prediction, traces[:, t])
```

```
key_byte = argmax_k(max_t(abs(score[k, t])))
```

Template attacks et attaques profilees

Les template attacks supposent un appareil de profilage similaire sur lequel l'attaquant connaît les clés. Il apprend alors la distribution statistique des traces pour chaque classe, puis compare les traces de la cible. Le papier d'Oswald et Mangard montre que le masking simple peut être contourné par des attaques profilées, parfois avec très peu de traces d'attaque.

Attaque	Besoin	Force	Limite
DPA	Traces cible, textes connus	Simple, historique	Ghost peaks, modèle faible
CPA	Modèle HW/HD	Robuste, très utilisée	Dépend du point de fuite
Template	Appareil de profilage	Quasi optimale statistiquement	Profilage coûteux
Deep learning	Dataset étiqueté	Supporte bruit/désynchronisation	Risque de surapprentissage

8. Fault injection et attaques par fautes

Une attaque par faute modifie volontairement le calcul : glitch d'horloge, glitch de tension, impulsion electromagnetique, laser, temperature, perturbation memoire. Le but est de provoquer une erreur utile : bit flip, instruction skip, comparaison contournee, sortie cryptographique fautive.

Modele Boneh-DeMillo-Lipton

Le papier de Boneh, DeMillo et Lipton montre qu'une signature RSA calculee avec CRT devient dangereuse si une seule partie du calcul modulo `p` ou `q` est fautive. Avec une signature correcte et une signature fautive, on peut retrouver un facteur du module par un PGCD.

Idee RSA-CRT fault : si S est correct et S' fautif seulement modulo p ou q , alors $\gcd(S - S', N)$ peut reveler p ou q . La cle privee tombe ensuite.

Fault attacks sur AES

Sur AES, une faute injectee vers les derniers tours peut etre propagee de maniere structuree. En comparant chiffre correct et chiffre fautif, une Differential Fault Analysis peut reduire fortement l'espace de cle. Les fautes peuvent aussi servir a sauter une verification de signature dans un bootloader.

Types de fautes

Type	Effet	Exemple	Defense
Bit flip	Un bit d'etat change	Registre RSA, etat AES	Code correcteur, redondance, verification
Instruction skip	Une instruction n'est pas executee	Bypass <code>if signature_ok</code>	Double verification, controle de flux durci
Data corruption	Bus ou memoire corrompu	Lecture flash perturbee	MAC, ECC memoire, CRC securise

Type	Effet	Exemple	Defense
Clock/voltage glitch	Timing viole	CPU execute mal	Moniteurs tension/horloge, reset securise

```
// Idee de verification defensive RSA-CRT
signature = rsa_crt_sign(message, private_key)
if rsa_verify(message, signature, public_key) != OK:
    zeroize_sensitive_state()
    return ERROR
return signature
```

9. Debug ports, boot ROM et secrets

En embarque, un port de debug oublie peut valoir plus qu'une faille crypto. JTAG, SWD, UART boot console, USB recovery, SPI flash externe, I2C EEPROM et pins strap de boot doivent etre consideres comme surface d'attaque.

Risques courants

- JTAG/SWD actif en production : lecture RAM, halt CPU, dump flash, modification registres.
- UART console root sans mot de passe ou avec mot de passe par default.
- Boot mode force par pin : demarrage sur USB/UART non authentifie.
- Flash externe non chiffree : extraction directe avec clip SOIC.
- Secrets dans variables d'environnement, device tree, scripts init, logs ou partition de configuration.

Bonnes pratiques

Element	Mesure	Point de vigilance
JTAG/SWD	Desactiver, verrouiller par fuse, debug authentifie	Prevoir un mode RMA securise
UART	Supprimer shell, proteger logs, pas de secrets	Ne pas laisser bootloader interactif
Flash	Chiffrement + authentication	Les cles ne doivent pas etre dans la flash
Boot pins	Straps proteges, modes limites	Tester les chemins de recovery
RMA	Effacement secrets avant retour usine	Journaliser sans exposer les cles

10. Secure boot, measured boot et mise a jour

Le secure boot construit une chaine de confiance depuis une racine immuable, souvent en ROM. Chaque etape verifie la suivante avant execution : ROM verifie bootloader, bootloader verifie kernel, kernel verifie initramfs/modules/applications critiques.

Boot ROM

Bootloader signe

Kernel / DTB

RootFS / App

Secure boot vs measured boot

Mecanisme	But	Exemple
Secure boot	Bloquer l'execution non autorisee	Verification ECDSA/RSA avant jump
Measured boot	Mesurer et journaliser l'etat de boot	Hash dans TPM PCR pour attestation
Verified boot	Verifier partitions et rootfs	dm-verity, AVB, FIT images signees
Anti-rollback	Refuser un firmware ancien vulnerable	Version monotone en eFuse/OTP/secure counter

Pipeline de mise a jour securisee

1. Construire le firmware dans un environnement controle.
2. Generer hash et metadata : version, produit, compatibilite, date, dependances.

3. Signer avec une cle de release protegee, idealement dans HSM ou service de signature.
4. Distribuer via canal TLS, mais ne pas dependre uniquement de TLS.
5. Verifier signature, version, produit cible et integrite sur l'appareil.
6. Installer en A/B slot ou avec rollback recovery.
7. Effacer secrets temporaires et journaliser le resultat.

```
# Pseudo-code de boot securise
image, metadata, signature = read_next_stage()
if metadata.product_id != DEVICE_PRODUCT_ID:
    halt_securely()
if metadata.version < monotonic_counter():
    halt_securely()
if verify_signature(public_key_hash_in_rom, image, metadata, signature) != OK:
    halt_securely()
jump_to(image.entry)
```

11. PUF, TRNG et stockage de clés

TRNG

Un True Random Number Generator tire son aléa d'un phénomène physique : bruit thermique, jitter d'oscillateur, métastabilité, bruit d'avalanche. En sécurité matérielle, le TRNG est critique parce qu'il alimente les clés, nonces, masques de side-channel et challenges.

Risques TRNG

- Biais ou corrélation dans la source physique.
- Blocage silencieux : la source ne produit plus d'entropie mais le système continue.
- Influence externe : température, tension, EM, vieillissement.
- Post-traitement mal compris : un hash ne crée pas d'entropie si la source est vide.

PUF

Une Physical Unclonable Function utilise les variations de fabrication pour produire une réponse propre à un composant. Exemple : SRAM PUF, ring oscillator PUF, arbiter PUF. Une PUF ne remplace pas toute la gestion de clés, mais elle peut aider à dériver une clé locale sans stocker directement cette clé en mémoire non volatile.

PUF	Principe	Avantage	Limite
SRAM PUF	État au démarrage de cellules SRAM	Souvent disponible sans circuit dédié	Besoin de correction d'erreurs
RO PUF	Comparer fréquences d'oscillateurs	Simple en FPGA/ASIC	Sensible à température/tension
Arbiter PUF	Comparer délais de chemins	Challenge-réponse riche	Modélisable par ML si non renforcée

Stockage de clés

- OTP/eFuse : bon pour hash de clé publique, compteur anti-rollback, configuration irréversible.
- Secure element/TPM : stockage isolé, opérations crypto internes, attestation.

- TEE/TrustZone : separation logiciel monde securise/normal, utile mais pas magique contre fautes physiques.
- Key ladder : deriver des cles de session sans exposer la cle racine.

12. FPGA, ASIC, DFT et scan chains

Les FPGA et ASIC ajoutent des risques propres : vol d'IP, bitstream copie, netlist reverse-engineering, insertion de trojan, scan chains qui exposent les registres internes, et differences entre simulation RTL et silicium final.

FPGA

Risque	Description	Defense
Bitstream theft	Copie du design depuis flash externe	Bitstream encryption + authentication
Readback	Lecture configuration interne	Desactiver readback, verrouillage securise
Partial reconfiguration	Chargement region modifiee	Signature par region, politique stricte
IP piracy	Clone ou reutilisation non autorisee	Watermarking, obfuscation, licensing hardware

ASIC et DFT

Le Design-for-Test est indispensable pour produire des circuits fiables, mais il peut exposer l'etat interne. Une scan chain mal protegee peut permettre de charger ou lire des registres sensibles, y compris des clees ou etats intermediaires d'AES.

Defenses scan/DFT

- Desactiver scan en mode production ou l'authentifier.
- Exclure les registres sensibles des scan chains.
- Chiffrer/compresser les patterns de test sensibles.
- Effacer les secrets avant passage en mode test.
- Contrôler les transitions test/normal par fuse et lifecycle state.

13. Hardware trojans

Un hardware trojan est une modification malveillante ou non autorisée d'un circuit. Il peut être inséré dans RTL, netlist, IP tierce, fab, layout ou même via un composant remplace. Il contient généralement un déclencheur et une charge utile.

Taxonomie

Axe	Categories	Exemples
Insertion	Specification, RTL, gate-level, layout, fabrication	IP modifiée, cellule ajoutée, masque altéré
Déclencheur	Always-on, rare event, compteur, séquence, analogique	Payload active après une suite d'entrées rares
Payload	Leakage, dégradation, denial, privilège	Fuite de clé, erreur AES, kill switch
Taille	Petit, distribué, paramétrique	Quelques portes ou modification de délais

Détection

- Analyse fonctionnelle : génération de tests pour activer des conditions rares.
- Analyse structurelle : features RTL/netlist, graphes, fan-in/fan-out, signaux rares.
- Side-channel : comparaison de puissance, EM, timing entre golden chip et suspect.
- Golden-free : détection statistique sans référence parfaite.
- Machine learning : classification de netlists, traces ou features physiques.

Piste locale dans ton workspace : le dossier `data/raw/ieee_dataport/` contient des traces de puissance AES pour des benchmarks avec états `TrojanDisabled`, `TrojanEnabled` et `TrojanTriggered`. Cela permet de faire un projet ML de détection hardware trojan par side-channel sans oscilloscope.

```
# Idée de pipeline ML pour traces hardware trojan
1. Charger les CSV de puissance par classe
2. Normaliser chaque trace
```

3. Extraire features : moyenne, variance, energie, RMS, FFT, pics
4. Entraîner SVM / RandomForest / CNN 1D
5. Evaluer : accuracy, F1-score, matrice de confusion
6. Expliquer les regions temporelles les plus discriminantes

14. Linux embarque et supply chain

Le PDF *The Trojan Horse in Your Code* rappelle un point tres important pour l'embarque moderne : une partie du risque ne vient pas du code que l'on ecrit, mais du code que l'on integre. Un firmware Linux contient bootloader, kernel, modules, bibliotheques, services, paquets Python/Node/Rust/C, scripts de build, images Docker, toolchains et dependances transitives.

Risque des dependances transitives

- Une dependance directe peut importer des dizaines de sous-dependances invisibles.
- Une vulnerabilite peut apparaitre dans du code embarque depuis des annees.
- Mettre a jour peut casser certification, drivers, ABI, tests terrain ou compatibilite produit.
- Les dependances orphelines restent dans les images meme si upstream ne maintient plus.

Controles pratiques pour Linux embarque

Controle	Role	Exemple
SBOM	Inventaire logiciel	SPDX/CycloneDX pour chaque firmware
SCA	Detection CVE/licences	Scan CI sur paquets directs et transitifs
Reproducible builds	Verifier provenance	Meme source donne meme artefact
Signature artefacts	Authentifier build	Cosign/in-toto/SLSA selon environnement
Runtime hardening	Limiter impact exploit	seccomp, AppArmor/SELinux, namespaces
Patch strategy	Reduire fenetre d'exposition	Backports, maintenance LTS, A/B updates

Durcissement systeme

- Supprimer comptes par default, services inutiles, shells de debug et paquets de build.
- Monter rootfs en lecture seule si possible, separer donnees persistantes.
- Utiliser least privilege : systemd sandboxing, capabilities minimales, seccomp.

- Activer protections memoire : ASLR, NX, stack canaries, RELRO, PIE.
- Journaliser sans exposer secrets : jamais de cle, token, nonce sensible dans logs.
- Prevoir rotation de certificats et fin de vie securisee.

15. Contre-mesures globales

Contre timing/cache

- Code constant-time : pas de branchement dependant d'un secret.
- Pas d'accès memoire indexe par secret : eviter tables AES sur cache partage.
- Utiliser AES-NI ou accélérateurs valides quand disponibles.
- Comparaisons constantes pour tags, MAC et signatures.

Contre power/EM

- Masking : separer les valeurs sensibles en parts aleatoires.
- Hiding : bruit, randomisation temporelle, shuffling, equilibrage logique.
- Design materiel : dual-rail/precharge, routage symetrique, decouplage, blindage.
- Evaluation : TVLA, CPA de verification, success rate, guessing entropy.

Contre fault injection

- Redondance temporelle : recalculer et comparer.
- Redondance spatiale : deux chemins differents ou co-processeur de verification.
- Detecteurs tension/horloge/temperature/lumiere.
- Control-flow integrity leger pour bootloader.
- Zeroization en cas d'anomalie et etat d'erreur sur.

Contre hardware trojans

- IP provenance, revue RTL, lint securite, analyse de signaux rares.
- Tests de declenchement rare et verification formelle quand possible.
- Side-channel fingerprinting en production ou qualification.
- Diversification de fournisseurs et controle chaine logistique.
- Design-for-security : logique de monitoring, assertions, compteurs d'anomalies.

Contre supply chain logicielle

- SBOM pour chaque release, y compris dependances transitives.
- Build reproductible et signe, controle des toolchains.

- Pinning de versions, verification de hash, miroirs internes.
- Politique de patch : CVE critiques, backports, fenetre d'exposition.
- Isolation runtime : meme si une dependance est vulnerable, reduire ce qu'elle peut faire.

Regle d'or : une contre-mesure doit etre testee contre l'attaque qu'elle pretend bloquer. Le bruit ne prouve pas l'absence de fuite, le secure boot ne prouve pas l'absence de faille runtime, et le masking mal implemente peut rester vulnerable aux attaques d'ordre superieur.

16. Projets sans materiel

Projet A : CPA AES avec traces publiques

But : recuperer un octet puis une cle AES avec CPA sur un dataset public tel que ASCAD, DPA Contest ou exemples ChipWhisperer.

1. Charger traces, textes clairs et metadata.
2. Tracer quelques signaux et verifier alignement/bruit.
3. Choisir un octet cible et le modele $HW(Sbox[P \oplus K])$.
4. Calculer les correlations pour 256 hypotheses.
5. Mesurer le nombre de traces necessaire pour obtenir le bon rang.
6. Comparer avec traces bruitees, desynchronisees ou masquees.

Projet B : Detection de hardware trojan par traces locales

But : utiliser `data/raw/ieee_dataport/` pour classifier des traces AES selon l'etat du trojan : disabled, enabled, triggered.

1. Construire un index des CSV et labels depuis les noms de dossiers.
2. Extraire features temporelles et frequentiellles.
3. Comparer SVM, RandomForest, XGBoost et CNN 1D.
4. Faire validation croisee par benchmark AES-T400/T500/T600/etc.
5. Etudier domain shift : entrainer sur un trojan, tester sur un autre.
6. Conclure sur detection golden-free et limites.

Projet C : Audit secure boot Linux embarque

1. Cartographier boot ROM, bootloader, kernel, rootfs, update.
2. Verifier signatures, anti-rollback, cles, modes recovery.
3. Tester les ports debug et consoles.
4. Construire une SBOM et scanner les CVE.
5. Proposer une architecture cible : A/B update, dm-verity, cle en OTP/TPM.

Projet D : Simulation fault injection

1. Coder une verification de signature simplifiee.
2. Simuler instruction skip sur la condition de verification.
3. Ajouter double verification, variable complementaire et etat d'erreur.
4. Mesurer quels patterns de fautes restent dangereux.

17. Checklist d'audit securite materielle

Architecture

- Quels sont les secrets ? Ou sont-ils generes, stockes, utilises, detruits ?
- Quelle est la racine de confiance ? ROM, OTP, TPM, secure element ?
- Quelles operations dependent de secrets ? Sont-elles constant-time ?
- Quelles interfaces existent en production ? JTAG, SWD, UART, USB recovery, Ethernet, radio ?

Firmware

- Le firmware est-il signe ? Chiffre si necessaire ? Anti-rollback ?
- Les clees privees de signature sont-elles hors du repo et protegees ?
- Existe-t-il un mode recovery authentifie ?
- Le rootfs est-il verifie et minimal ?

Cryptographie

- AES utilise-t-il un mode authentifie avec nonce unique ?
- RSA-CRT verifie-t-il le resultat avant sortie ?
- ECDSA protege-t-il strictement son nonce ?
- Le TRNG a-t-il health tests et fallback sur DRBG correctement seed ?

Side-channel/fault

- Les implementations critiques ont-elles ete testees TVLA ?
- Les zones sensibles sont-elles masquees ou rendues constantes ?
- Les detecteurs tension/horloge provoquent-ils un reset sur ?
- Les erreurs effacent-elles les secrets temporaires ?

Supply chain

- Existe-t-il une SBOM par version de firmware ?
- Les dependances transitives sont-elles inventoriees ?
- Les artefacts de build sont-ils signes et reproductibles ?

- Les composants EOL ont-ils une strategie de mitigation ?

18. Notes de lecture des PDF utilises

Les PDF du dossier `C:\Users\ngdia\Documents\doc_linuc_embarquee` ont ete utilises comme corpus local. Les idees ci-dessous sont reformulees et integrees au cours, sans reproduction longue des textes.

PDF	Contenu lu	Parties ajoutees au cours
Understanding Cryptography by Christof Paar .pdf	Fondations crypto : DES, AES, modes, RSA, DH, ECC, signatures, hash, key establishment.	Sections crypto, DES/AES, RSA/ECC, modes, signatures, DH, hachage, secure boot et gestion de cles.
3-540-48405-1_25.pdf	Kocher/Jaffe/Jun, Differential Power Analysis : SPA, DPA, traces de puissance, DES, bruit, contre-mesures.	Sections SPA/DPA, methode d'attaque, bruit, selection function, contre-mesures power.
3-540-68697-5_9.pdf	Kocher, timing attacks sur RSA, DH, DSS : temps variable, detection statistique, blinding.	Sections timing attacks, RSA/DH, constant-time, limites des delais aleatoires.
978-3-540-28632-5_2.pdf	Brier/Clavier/Olivier, CPA : modele distance de Hamming, correlation, ghost peaks DPA, DES/AES.	Section CPA avec modele $W = aH(D \oplus R) + b$, comparaison DPA/CPA, pseudo-code CPA AES.
3-540-69053-0_4.pdf	Boneh/DeMillo/Lipton, attaques par fautes sur RSA-CRT, Rabin, protocoles d'identification.	Section fault injection, RSA-CRT PGCD, instruction skip, verification defensive.
11967668_16.pdf	Oswald/Mangard, template attacks sur masking AES, attaque profilee contre implementation masquee.	Section template attacks, limites du masking premier ordre, besoin d'evaluation avantee.
The-Trojan-Horse-in-Your-Code.pdf	Risque herite des dependances open source, dependances transitives, composants orphelins, durcissement runtime.	Section Linux embarque et supply chain : SBOM, SCA, patching, runtime hardening, dependances transitives.

Bibliographie conseillée pour aller plus loin

- Stefan Mangard, Elisabeth Oswald, Thomas Popp, *Power Analysis Attacks*.
- Mark Tehranipoor, Cliff Wang, *Introduction to Hardware Security and Trust*.
- Christof Paar, Jan Pelzl, *Understanding Cryptography*.
- Kocher, Jaffe, Jun, *Differential Power Analysis*.
- Brier, Clavier, Olivier, *Correlation Power Analysis with a Leakage Model*.
- Boneh, DeMillo, Lipton, *On the Importance of Checking Cryptographic Protocols for Faults*.
- Documentation ChipWhisperer, datasets ASCAD ANSSI, DPA Contest et jeux de traces académiques.
- Standards utiles : FIPS 197 pour AES, FIPS 186-5 pour signatures, FIPS 140-3 pour modules cryptographiques, ISO/IEC 17825 pour tests side-channel.

19. Resume final

La securite materielle relie trois mondes : le mathematique, le logiciel et le physique. Une cle AES peut etre solide, mais fuir dans une trace de puissance. Une signature RSA peut etre correcte, mais tomber a cause d'une faute. Un firmware peut etre chiffre, mais dangereux s'il n'est pas authentifie. Un systeme Linux peut etre propre dans son code maison, mais fragile par ses dependances.

Pour un master en securite des systemes embarques, l'objectif est de savoir construire un raisonnement complet : actif a proteger, modele de menace, primitive crypto, implementation, attaque possible, mesure experimentale, contre-mesure, validation, limites.

AES

RSA

ECC

Side-channel

Fault injection

Secure boot

PUF/TRNG

Hardware trojan

Linux embarque

Supply chain